

# EXPERIMENT - 15

## Title

Develop a Leave Management System using ReactJS

---

## Aim

To develop a ReactJS application that allows users to:

- Apply for **Casual Leave**
  - Apply for **Medical Leave**
  - View remaining leave balance dynamically
- 

## Prerequisites

- Basic knowledge of JavaScript (ES6)
  - Understanding of ReactJS components
  - Knowledge of `useState` hook
  - Node.js installed
- 

## Software Requirements

- OS: Windows / Linux / macOS
- Node.js (LTS)
- ReactJS
- VS Code

- Browser (Chrome)
- 

## Theory

React allows dynamic UI updates using state management.

### Concepts Used:

- Functional Components
- `useState` Hook
- Form Handling
- Conditional Rendering
- State Validation

In this experiment:

- Organization gives fixed leave balance
  - User applies leave
  - Balance updates automatically
  - Prevents applying more leave than available
- 

## Project Setup

### Step 1: Create React App

```
Shell
npm create vite@latest
cd leave-app
npm start
```

---

# Source Code

Replace `src/App.js` with:

JavaScript

```
import React, { useState } from "react";
import "../App.css";

function App() {
  const initialLeaveBalance = {
    CasualLeave: 12,
    MedicalLeave: 10,
    EarnedLeave: 8,
  };

  const [leaveBalance, setLeaveBalance] =
    useState(initialLeaveBalance);
  const [leaveHistory, setLeaveHistory] = useState([]);
  const [selectedLeaveType, setSelectedLeaveType] =
    useState("CasualLeave");
  const [leaveDays, setLeaveDays] = useState(1);

  const handleApplyLeave = () => {
    if (leaveDays < 1 || isNaN(leaveDays)) {
      alert("Please enter a valid number of days.");
      return;
    }

    if (leaveBalance[selectedLeaveType] >= leaveDays) {
      // Update leave balance
      setLeaveBalance({
        ...leaveBalance,
        [selectedLeaveType]:
```

```

        leaveBalance[selectedLeaveType] - leaveDays,
    });

    // Add to leave history
    setLeaveHistory([
        ...leaveHistory,
        {
            type: selectedLeaveType,
            days: leaveDays,
            date: new Date().toLocaleDateString(),
        },
    ]);

    alert("Leave applied successfully!");
} else {
    alert("Not enough leave balance!");
}
};

const handleResetLeaves = () => {
    setLeaveBalance(initialLeaveBalance);
    setLeaveHistory([]);
};

return (
    <div className="App">
        <h1>Leave Management System</h1>

        {/* Leave Balance Section */}
        <div className="leave-balance">
            <h2>Leave Balance</h2>
            <ul>

```

```

        {Object.entries(leaveBalance).map(([type, days]) => (
            <li key={type}>
                {type}: {days} days
            </li>
        ))}
    </ul>
</div>

{/* Apply Leave Section */}
<div className="apply-leave">
    <h2>Apply Leave</h2>

    <label>
        Leave Type:
        <select
            value={selectedLeaveType}
            onChange={(e) => setSelectedLeaveType(e.target.value)}
        >
            {Object.keys(leaveBalance).map((type) => (
                <option key={type} value={type}>
                    {type}
                </option>
            ))}
        </select>
    </label>

    <label>
        Number of Days:
        <input
            type="number"
            min="1"
            value={leaveDays}

```

```

        onChange={(e) =>
            setLeaveDays(parseInt(e.target.value) || 0)
        }
    />
</label>

<button onClick={handleApplyLeave}>Apply Leave</button>
</div>

{/* Leave History Section */}
<div className="leave-history">
    <h2>Leave History</h2>

    {leaveHistory.length > 0 ? (
        <ul>
            {leaveHistory.map((leave, index) => (
                <li key={index}>
                    {leave.type}: {leave.days} day(s) on {leave.date}
                </li>
            ))}
        </ul>
    ) : (
        <p>No leave history available.</p>
    )}
</div>

<button
    className="reset-button"
    onClick={handleResetLeaves}
>
    Reset Leave Balances
</button>

```

```
    </div>
  );
}

export default App;
```

---

## App.css

Replace `src/App.css` with:

```
CSS

.App {
  text-align: center;
  font-family: Arial, sans-serif;
  padding: 20px;
}

h1 {
  font-size: 2em;
  margin-bottom: 20px;
}

.leave-balance,
.apply-leave,
.leave-history {
  margin: 20px 0;
  padding: 20px;
  border: 1px solid #ddd;
```

```
border-radius: 5px;  
background-color: #f9f9f9;  
}
```

```
h2 {  
  margin-bottom: 10px;  
}
```

```
ul {  
  list-style: none;  
  padding: 0;  
}
```

```
li {  
  margin: 5px 0;  
}
```

```
label {  
  display: block;  
  margin: 10px 0;  
}
```

```
input,  
select {  
  margin-left: 10px;  
  padding: 5px;  
  font-size: 16px;  
}
```

```
button {  
  margin-top: 10px;  
  padding: 10px 15px;
```



```
font-size: 16px;
background-color: #007bff;
color: white;
border: none;
border-radius: 5px;
cursor: pointer;
transition: background 0.3s;
}

button:hover {
  background-color: #0056b3;
}

.reset-button {
  background-color: #dc3545;
}

.reset-button:hover {
  background-color: #c82333;
}
```

---

## Execution Steps

Run project:

Shell

```
npm start
```

2 Open browser:

None

<http://localhost:3000>

---

## Functionality Demonstrated

| Feature         | Description               |
|-----------------|---------------------------|
| Leave Balance   | Displays available leave  |
| Apply Leave     | Reduces leave dynamically |
| Validation      | Prevents negative balance |
| Message Display | Shows approval or error   |

---

## Sample Output

Initial:

None

Casual Leave: 10

Medical Leave: 8

After applying 3 Casual Leave:

None

Casual Leave: 7

Message: Casual Leave Approved!

If applying more than balance:

None

Message: Not enough Casual Leave balance.

---

## Expected Result

The Leave Management System successfully manages leave balances and dynamically updates available leave using ReactJS state management.

---

## Result

Thus, a Leave Management System was successfully developed using ReactJS.

---

## Viva Voce Questions

1. What is useState?
  2. Why do we use parseInt?
  3. How does React re-render?
  4. What is event handling in React?
  5. How can this connect to backend?
  6. How to persist leave data?
-

## Conclusion

This experiment demonstrates ReactJS state management and dynamic UI updates by implementing a Leave Management System.